Scaling the Memory Wall

Towards 3D-DRAM-based Accelerators for Efficient Generative Inference †

Prashant J. Nair *

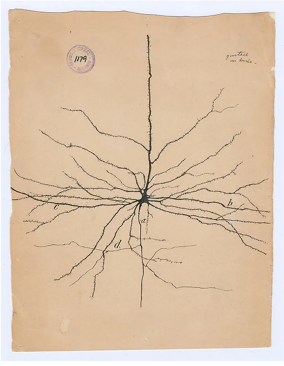
The University of British Columbia and d-Matrix Inc.

* Also affiliated as an Associate Professor at the University of British Columbia (UBC).

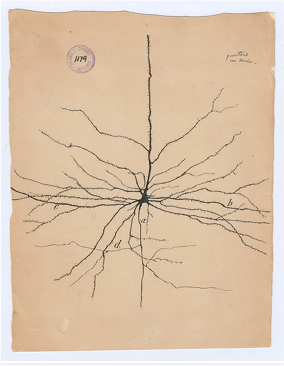
† Work done during sabbatical/leave from UBC.

28th June 2026

Year 1888: When the Brain Revealed Its Wiring



Year 1888: When the Brain Revealed Its Wiring



Santiago Ramón y Cajal's drawing of pyramidal neurons in the rabbit cerebral cortex (ca. 1888–1899).

Each neuron stores, processes, and communicates, with wires measured in *microns*.

The Lesson in the Drawing

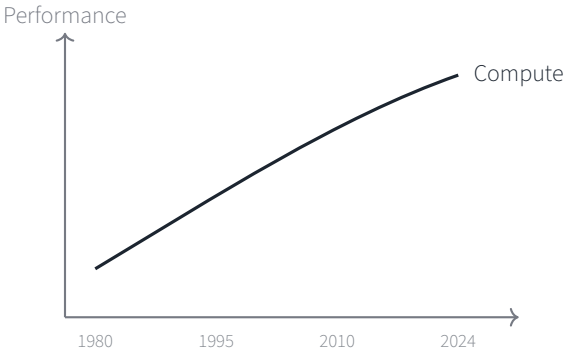
Santiago Ramón y Cajal proved that the brain is not a continuous network, but *discrete cells* with short, local connections → overturning a century of doctrine.

The Lesson in the Drawing

Santiago Ramón y Cajal proved that the brain is not a continuous network, but *discrete cells* with short, local connections → overturning a century of doctrine.

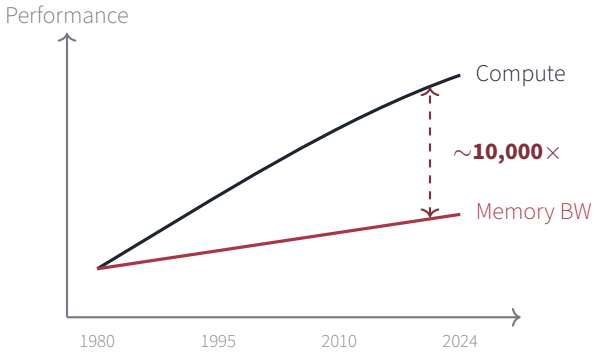
Nature solved the bandwidth problem not with faster signals,
but with **shorter wires.**

A Gap That Grew



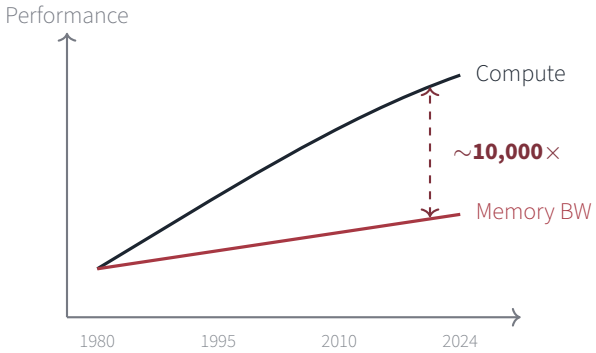
- Processor speed: ∼60%/year

A Gap That Grew



- Processor speed: ∼60%/year
- DRAM bandwidth: ∼7%/year
- Caches survived by exploiting *data reuse*

A Gap That Grew



- Autoregressive decode eliminates reuse.

Autoregressive decode reads every weight, every token.

⇒ **there is no reuse to exploit.**

What Changed in the Last Five Years

Before: Training & Classification

- Fixed input sizes, large batches
- High reuse across samples
- Compute-bound: attention scales as $O(L^2)$
- ✓ GPUs excel here

Notation: L = **context length** (number of tokens in the prompt + tokens generated so far).

What Changed in the Last Five Years

Before: Training & Classification

- Fixed input sizes, large batches
- High reuse across samples
- Compute-bound: attention scales as $O(L^2)$
- ✓ GPUs excel here

Now: Interactive Generation

- Variable context, single-token steps
- Read *all* weights *every* token
- Memory-bound: KV-cache $\Rightarrow O(L)$ per token
- ✗ GPUs starve for data

Notation: L = **context length** (number of tokens in the prompt + tokens generated so far).

What Changed in the Last Five Years

Before: Training & Classification

- Fixed input sizes, large batches
- High reuse across samples
- Compute-bound: attention scales as $O(L^2)$
- ✓ GPUs excel here

Now: Interactive Generation

- Variable context, single-token steps
- Read *all* weights *every* token
- Memory-bound: KV-cache $\Rightarrow O(L)$ per token
- ✗ GPUs starve for data

Batch processing → interactive serving:
the bottleneck moved from compute to **memory.**

Notation: L = **context length** (number of tokens in the prompt + tokens generated so far).

The Decode Bottleneck

Two Phases, Different Bottlenecks

Prefill Phase

Workload: Process L input tokens in parallel

Compute: $O(L^2 \cdot d)$ — self-attention dominates

Memory: $O(L \cdot d)$ KV writes

Reuse: High — same weights, many tokens

Bottleneck: ✓ Compute-bound

Notation: L = context length (tokens so far), d = hidden dimension.

Two Phases, Different Bottlenecks

Prefill Phase

Workload: Process L input tokens in parallel

Compute: $O(L^2 \cdot d)$ — self-attention dominates

Memory: $O(L \cdot d)$ KV writes

Reuse: High — same weights, many tokens

Bottleneck: ✓ Compute-bound

Decode Phase

Workload: Generate one token per step

Compute: $O(L \cdot d)$ — attention over full cache

Memory: Read *entire* model + KV cache

Reuse: Minimal — full read every token

Bottleneck: ✗ **Memory-bound**

Notation: L = context length (tokens so far), d = hidden dimension.

Two Phases, Different Bottlenecks

Prefill Phase

Workload: Process L input tokens in parallel

Compute: $O(L^2 \cdot d)$ — self-attention dominates

Memory: $O(L \cdot d)$ KV writes

Reuse: High — same weights, many tokens

Bottleneck: ✓ Compute-bound

Decode Phase

Workload: Generate one token per step

Compute: $O(L \cdot d)$ — attention over full cache

Memory: Read *entire* model + KV cache

Reuse: Minimal — full read every token

Bottleneck: ✗ **Memory-bound**

Even at a 50-50 token split, **∼90% of inference time is decode.**

Notation: L = context length (tokens so far), d = hidden dimension.

Quantifying the Memory Traffic

Llama-3.1 70B: KV Cache Growth Per Token

$$\underbrace{80}_{\text{layers}} \times \underbrace{8}_{\text{KV heads}} \times \underbrace{128}_{\text{head dim}} \times \underbrace{2}_{\text{K+V}} \times \underbrace{1\,\text{B}}_{\text{INT8}} = \textbf{0.16 MB / token / user}$$

Quantifying the Memory Traffic

Llama-3.1 70B: KV Cache Growth Per Token

$$\underbrace{80}_{\text{layers}} \times \underbrace{8}_{\text{KV heads}} \times \underbrace{128}_{\text{head dim}} \times \underbrace{2}_{\text{K+V}} \times \underbrace{1\,\text{B}}_{\text{INT8}} = \textbf{0.16 MB / token / user}$$

Per decode step, read: all weights (∼70 GB) + **KV cache** (0.16 MB × L per user).

Context (L)	**KV / User**	**Total Read / Token**†	**Time @ 18 TB/s**
8 K	1.25 GB	71.25 GB	3.96 ms
32 K	5 GB	75 GB	4.17 ms
128 K	20 GB	90 GB	5.00 ms

† Single user (batch = 1). Batched serving multiplies the KV read by concurrent users.

Quantifying the Memory Traffic

Llama-3.1 70B: KV Cache Growth Per Token

$$\underbrace{80}_{\text{layers}} \times \underbrace{8}_{\text{KV heads}} \times \underbrace{128}_{\text{head dim}} \times \underbrace{2}_{\text{K+V}} \times \underbrace{1\,\text{B}}_{\text{INT8}} = \textbf{0.16 MB / token / user}$$

Per decode step, read: all weights (∼70 GB) + **KV cache** (0.16 MB × L per user).

Context (L)	KV / User	Total Read / Token†	Time @ 18 TB/s
8 K	1.25 GB	71.25 GB	3.96 ms
32 K	5 GB	75 GB	4.17 ms
128 K	20 GB	90 GB	5.00 ms

† Single user (batch = 1). Batched serving multiplies the KV read by concurrent users.

Memory read time *alone* can consume a large fraction of typical TPOT budgets (**10–50 ms**).

How Memory-Bound Is Decode?

Arithmetic intensity $\Rightarrow$ compute per byte moved:

$$\mathrm{AI} = \frac{\text{FLOPs}}{\text{Bytes transferred}}$$

The **balance point** of an accelerator is:

$$\mathrm{AI}^* = \frac{P_{\text{peak}}}{B_{\text{mem}}}$$

If $\mathrm{AI} < \mathrm{AI}^* \Rightarrow$ memory-bound: more compute won't help.

How Memory-Bound Is Decode?

Arithmetic intensity $\Rightarrow$ compute per byte moved:

$$\mathrm{AI} = \frac{\text{FLOPs}}{\text{Bytes transferred}}$$

The **balance point** of an accelerator is:

$$\mathrm{AI}^* = \frac{P_{\text{peak}}}{B_{\text{mem}}}$$

If $\mathrm{AI} < \mathrm{AI}^* \Rightarrow$ memory-bound: more compute won't help.

Llama-70B decode (BS = 1, 32K):

- 70B params $\times$ 2 $=$ 140 GFLOPs / token
- 70 GB weights $+$ 5 GB KV $=$ 75 GB / token
- $\Rightarrow$ AI $=$ 1.9 FLOPs/byte

How Memory-Bound Is Decode?

Arithmetic intensity ⇒ compute per byte moved:

$$\mathrm{AI} = \frac{\text{FLOPs}}{\text{Bytes transferred}}$$

The **balance point** of an accelerator is:

$$\mathrm{AI}^* = \frac{P_{\text{peak}}}{B_{\text{mem}}}$$

If $\mathrm{AI} < \mathrm{AI}^* \Rightarrow$ memory-bound: more compute won't help.

Llama-70B decode (BS = 1, 32K):

- 70B params × 2 = 140 GFLOPs / token
- 70 GB weights + 5 GB KV = 75 GB / token
- ⇒ AI = 1.9 FLOPs/byte

Where does AI* sit?

All rows use 5 PFLOPS compute:

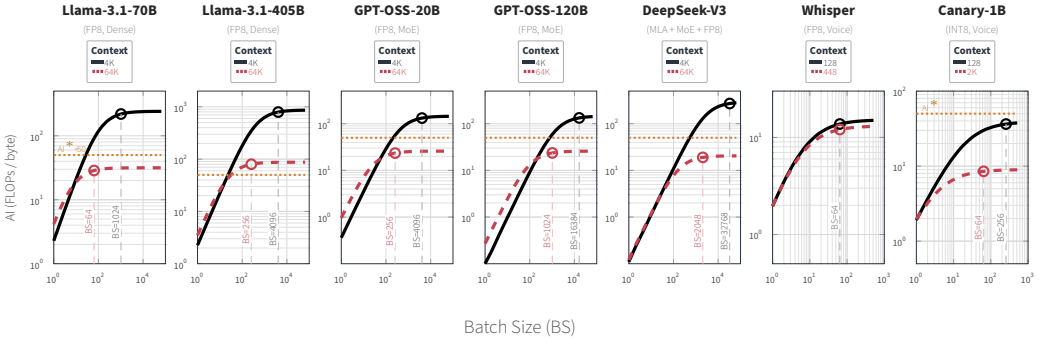
Memory	B_{mem}	AI*	Gap†
SRAM	150 TB/s	33	17×
3D-DRAM	**100 TB/s**	**50**	**26×**
HBM	18 TB/s	278	146×

† AI* /AI, where AI ≈ 1.9 at BS = 1.

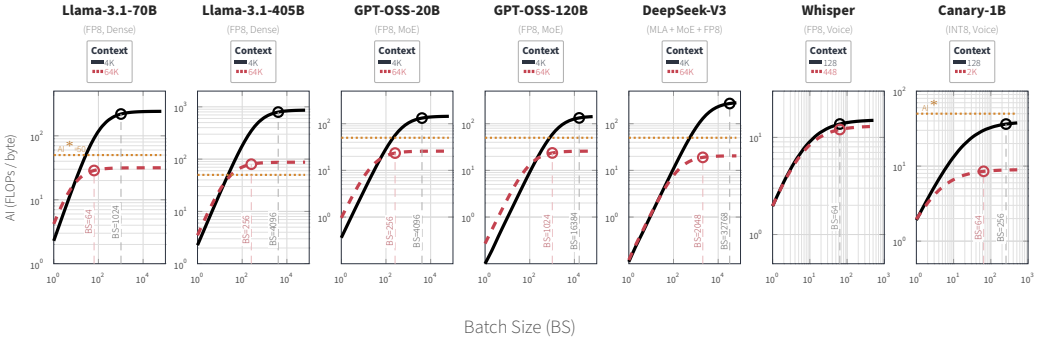
$\mathrm{AI} \approx 2 \ll \mathrm{AI}^* \approx 33$–$278$

Decode sits **15–140× below the roofline knee.**

It is Not Just One Model



It is Not Just One Model



Dense, MoE, and voice models — **all sit below AI* = 50** at realistic batch sizes.
Longer contexts flatten the curves further. *Could larger batches close the gap?*

Batching Helps, But Has Hard Limits

Batching amortizes weight reads:

- Batch B users together
- Read weights once, reuse across B requests
- AI scales: $\mathrm{AI}_B \approx B \cdot \mathrm{AI}_1$

But KV cache scales *with* batch size:

- Each user carries its own KV cache
- Total memory: $\underbrace{70\,\mathrm{GB}}_{\text{weights}} + B \times \underbrace{\mathrm{KV}}_{\text{per user}}$
- At 128K, $B = 8$: $70 + 8 \times 20 = 230\,\mathrm{GB}$

Batching Helps, But Has Hard Limits

Batching amortizes weight reads:

- Batch B users together
- Read weights once, reuse across B requests
- AI scales: $\mathrm{AI}_B \approx B \cdot \mathrm{AI}_1$

But KV cache scales *with* batch size:

- Each user carries its own KV cache
- Total memory: $\underbrace{70\,\mathrm{GB}}_{\text{weights}} + B \times \underbrace{\mathrm{KV}}_{\text{per user}}$
- At 128K, $B = 8$: $70 + 8 \times 20 = 230$ GB

Realistic Batch Sizes

Poisson arrivals simulated over 1 second:

Arrival rate	Max batch	Avg batch
110 req/s	1	1.0
500 req/s	5	1.2
1000 req/s	27	5.1

Even under heavy load, batch stays < 32.

Hard limits: KV-cache capacity, latency SLAs, bursty arrivals.

A **26× gap** (3D-DRAM) needs BS ≥ 26 to close $\Rightarrow$ but real batches average **∼5.**
The memory wall stands.

The decode bottleneck is fundamental. Batching cannot solve it. Parallelism taxes it further.

The question becomes:

What kind of memory do we actually need?

Fast enough to feed the compute. Large enough to hold the model.

Today’s Memory Technologies

SRAM: Bandwidth Advantage

Why SRAM is fast:

- 6T cell with direct bit-line access
- On-die: no interposer, no package crossing
- Sub-nanosecond access latency

Specs (on-die, per card):

Bandwidth	**150 TB/s**
Capacity	**4 GB**
Latency	∼1 ns
I/O energy	∼0.5 pJ/bit

SRAM: Bandwidth Advantage

Why SRAM is fast:

- 6T cell with direct bit-line access
- On-die: no interposer, no package crossing
- Sub-nanosecond access latency

Specs (on-die, per card):

Bandwidth	**150 TB/s**
Capacity	**4 GB**
Latency	∼1 ns
I/O energy	∼0.5 pJ/bit

Why SRAM cannot scale:

- 6T cell is **100–150**× **larger** than DRAM 1T1C
- Leakage: ∼1 nW/bit (significant at GB scale)
- Practical limit: ∼**4 GB** per card
- Cost: ∼$1000/GB vs. ∼$10/GB for DRAM

Llama-70B on SRAM

70 GB weights + KV cache ÷ 4 GB/card

⇒ TP = 8, PP = 4 = **32 cards**

(163 collective ops every token)

SRAM: Bandwidth Advantage

Why SRAM is fast:

- 6T cell with direct bit-line access
- On-die: no interposer, no package crossing
- Sub-nanosecond access latency

Specs (on-die, per card):

Bandwidth	**150 TB/s**
Capacity	**4 GB**
Latency	∼1 ns
I/O energy	∼0.5 pJ/bit

Why SRAM cannot scale:

- 6T cell is **100–150× larger** than DRAM 1T1C
- Leakage: ∼1 nW/bit (significant at GB scale)
- Practical limit: ∼**4 GB** per card
- Cost: ∼$1000/GB vs. ∼$10/GB for DRAM

Llama-70B on SRAM

70 GB weights + KV cache ÷ 4 GB/card

⇒ TP = 8, PP = 4 = **32 cards**

(163 collective ops every token)

Very fast memory — but **4 GB cannot hold a 70B model.**

HBM: Capacity Advantage

Why HBM has capacity:

- 1T1C DRAM cell (dense storage)
- 3D stacking: 8–12 dies per stack
- Wide bus: 1024-bit × 8 channels

Specs (HBM3/3e, per package):

Capacity	**192 GB**
Bandwidth	**18 TB/s** (6 stacks)
Latency	∼20–30 ns
I/O energy	∼2–3 pJ/bit

HBM: Capacity Advantage

Why HBM has capacity:

- 1T1C DRAM cell (dense storage)
- 3D stacking: 8–12 dies per stack
- Wide bus: 1024-bit × 8 channels

Specs (HBM3/3e, per package):

Capacity	**192 GB**
Bandwidth	**18 TB/s** (6 stacks)
Latency	∼20–30 ns
I/O energy	∼2–3 pJ/bit

Why HBM bandwidth is limited:

- 2.5D: memory *beside* logic on interposer
- Horizontal mm-scale routing
- Bump pitch: 55 μm limits I/O density
- PHY overhead at each stack interface

Llama-70B on HBM

75 GB read/token ÷ 18 TB/s = **4.2 ms**

That's 8–42% of a 10–50 ms TPOT budget spent *just* on memory reads.

HBM: Capacity Advantage

Why HBM has capacity:

- 1T1C DRAM cell (dense storage)
- 3D stacking: 8–12 dies per stack
- Wide bus: 1024-bit × 8 channels

Specs (HBM3/3e, per package):

Capacity	**192 GB**
Bandwidth	**18 TB/s** (6 stacks)
Latency	∼20–30 ns
I/O energy	∼2–3 pJ/bit

Why HBM bandwidth is limited:

- 2.5D: memory *beside* logic on interposer
- Horizontal mm-scale routing
- Bump pitch: 55 μm limits I/O density
- PHY overhead at each stack interface

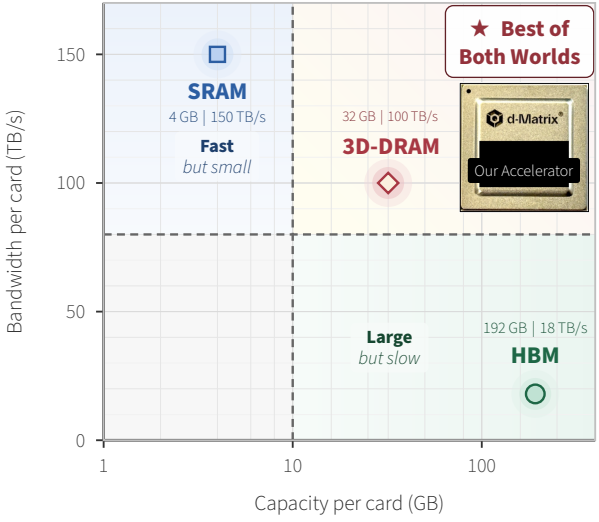
Llama-70B on HBM

75 GB read/token ÷ 18 TB/s = **4.2 ms**

That's 8–42% of a 10–50 ms TPOT budget spent *just* on memory reads.

Plenty of capacity — but **18 TB/s leaves the accelerator starving.**

The Fundamental Trade-off

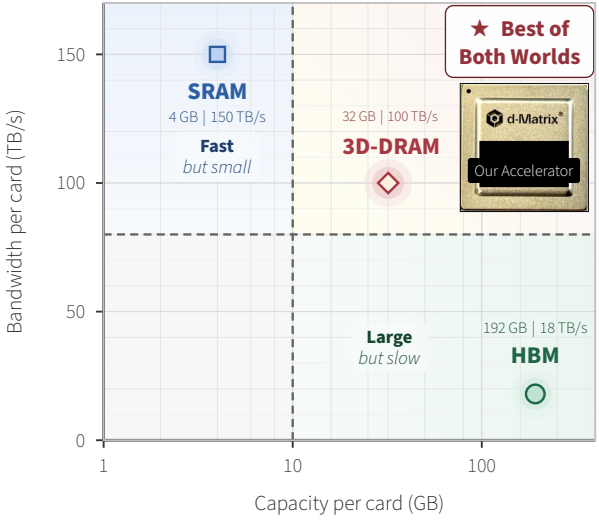


How do we reach the upper-right?

Stack memory *below* logic, face-to-face:

- Vertical μbumps (μm, not mm)
- $C \propto L$: shorter wires $\Rightarrow$ lower energy
- 36 μm pitch $\Rightarrow$ **700 conn/mm^2**
  (vs. 55 μm / $\sim$100 conn/mm^2 for HBM)
- More parallel wires, each at lower pJ/bit

The Fundamental Trade-off



How do we reach the upper-right?

Stack memory *below* logic, face-to-face:

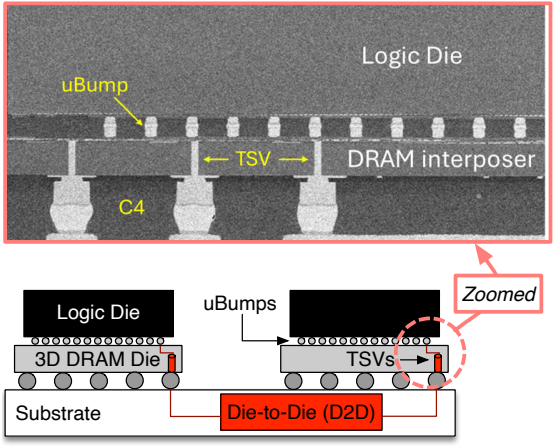
- Vertical μbumps (μm, not mm)
- $C \propto L$: shorter wires $\Rightarrow$ lower energy
- 36 μm pitch $\Rightarrow$ **700 conn/mm^2**
  (vs. 55 μm / $\sim$100 conn/mm^2 for HBM)
- More parallel wires, each at lower pJ/bit

The Opportunity

3D face-to-face stacking delivers **7× denser interconnect** and **6–8× lower I/O energy** than HBM $\Rightarrow$ enough to reach 100 TB/s at 32 GB per card.

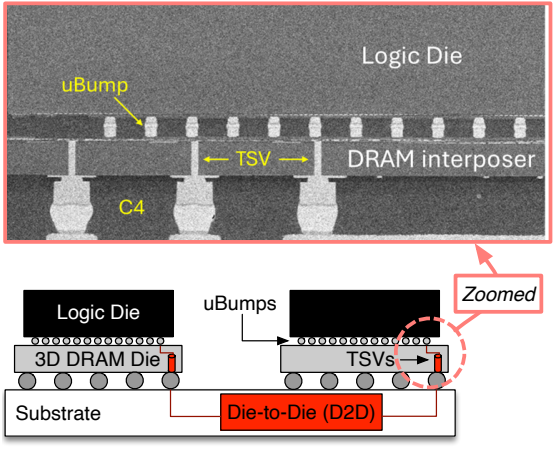
3D-DRAM: The Opportunity

The 3D-DRAM Accelerator



Face-to-face stack: TSMC N4P logic die bonded to 3D-DRAM interposer die.

The 3D-DRAM Accelerator

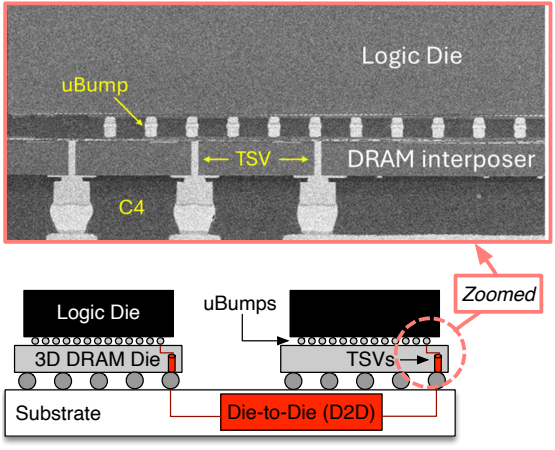


Face-to-face stack: TSMC N4P logic die bonded to 3D-DRAM interposer die.

Per-chiplet specs:

Compute	5 PFLOPS (FP8)
DRAM banks	840
Capacity	**32 GB** (3D-DRAM)
	+ 128 GB (LPDDR5X)
Bandwidth	**100 TB/s**
μBump pitch	36 μm (F2F)

The 3D-DRAM Accelerator



Face-to-face stack: TSMC N4P logic die bonded to 3D-DRAM interposer die.

Per-chiplet specs:

Compute	5 PFLOPS (FP8)
DRAM banks	840
Capacity	**32 GB** (3D-DRAM) + 128 GB (LPDDR5X)
Bandwidth	**100 TB/s**
μBump pitch	36 μm (F2F)

Llama-70B deployment:

	SRAM	3D-DRAM
Cards	**32**	**4**
TP / PP	8 / 4	4 / 1
Collectives	163	160

SRAM-like bandwidth. HBM-like capacity.
We built this.

The Integration Problem

3D-DRAM Is Not “HBM, But Closer”

The same technology class but *none* of the same assumptions.

3D-DRAM Is Not “HBM, But Closer”

The same technology class but *none* of the same assumptions.

	HBM (2.5D)	**3D-DRAM (F2F)**
Protocol	JEDEC standard	Custom single-cycle interface
Transfer	Multi-cycle burst (BL 4–16)	256-bit per bank per cycle
DBI / ECC	PHY-level, dedicated pins	**No burst ⇒ no DBI pin**
Thermals	Decoupled from logic	**Coupled to logic (T_j=105°C)**
Bank count	∼32 per stack	**840 per chiplet**

3D-DRAM Is Not “HBM, But Closer”

The same technology class but *none* of the same assumptions.

	HBM (2.5D)	3D-DRAM (F2F)
Protocol	JEDEC standard	Custom single-cycle interface
Transfer	Multi-cycle burst (BL 4–16)	256-bit per bank per cycle
DBI / ECC	PHY-level, dedicated pins	**No burst ⇒ no DBI pin**
Thermals	Decoupled from logic	**Coupled to logic (T_j=105°C)**
Bank count	∼32 per stack	**840 per chiplet**

Several assumptions behind conventional DRAM management **do not apply.**

The 3D-DRAM Integration Landscape

Hardware	Architecture	Co-Design
TSV density vs. routing congestion trade-offs	Bank-to-channel mapping: 840 banks $\neq 256 \times 4$	Thermal: T_j=105°C $\Rightarrow$ 8$\times$ faster refresh
Power delivery through 3D stack (IR drop, decap)	Weight & KV-cache tiling across banked memory	ECC, DBI metadata, & scrub share bank rows
Signal integrity: crosstalk & SSN on wide parallel bus	Redundancy without breaking channel symmetry	Refresh scheduling vs. compute pipeline bubbles
I/O power: no burst structure $\Rightarrow$ no PHY DBI	LPDDR5X as secondary tier: data placement policy	Known-good-die testing $\rightarrow$ post-package repair flow
μBump yield & alignment at 36 μm pitch	Compiler support for stream-aware access patterns	Multi-high stacking: routing, power, thermals
Clock distribution & skew across die stack	Runtime KV-cache alloc & eviction across banks	Hybrid bonding roadmap: $<$2 μm pitch, 8-high

The 3D-DRAM Integration Landscape

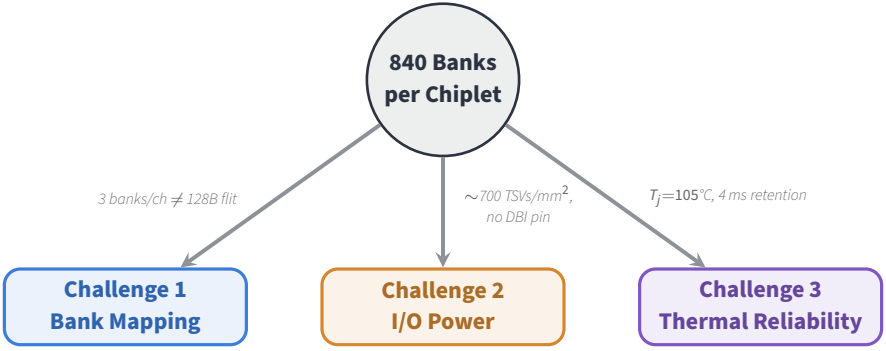
Hardware	Architecture	Co-Design
TSV density vs. routing congestion trade-offs	★ **Bank-to-channel mapping: 840 banks $\neq$ 256 $\times$ 4**	★ **Thermal:** T_j=105**°C** $\Rightarrow$ **8 $\times$ faster refresh**
Power delivery through 3D stack (IR drop, decap)	Weight & KV-cache tiling across banked memory	ECC, DBI metadata, & scrub share bank rows
Signal integrity: crosstalk & SSN on wide parallel bus	★ **Redundancy without breaking channel symmetry**	Refresh scheduling vs. compute pipeline bubbles
★ **I/O power: no burst structure $\Rightarrow$ no PHY DBI**	LPDDR5X as secondary tier: data placement policy	Known-good-die testing $\rightarrow$ post-package repair flow
μBump yield & alignment at 36 μm pitch	Compiler support for stream-aware access patterns	Multi-high stacking: routing, power, thermals
Clock distribution & skew across die stack	Runtime KV-cache alloc & eviction across banks	Hybrid bonding roadmap: $<$2 μm pitch, 8-high

The 3D-DRAM Integration Landscape

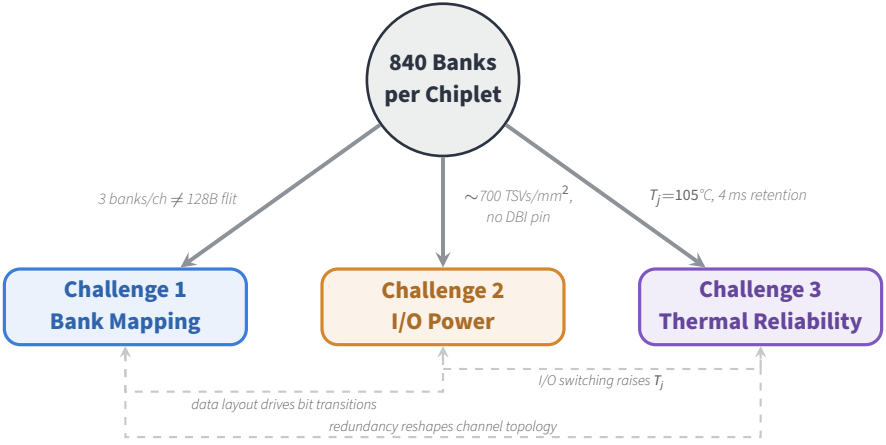
Hardware	Architecture	Co-Design
TSV density vs. routing congestion trade-offs	★ **Bank-to-channel mapping: 840 banks $\neq$ 256 $\times$ 4**	★ **Thermal:** T_j=105**°C** $\Rightarrow$ **8 $\times$ faster refresh**
Power delivery through 3D stack (IR drop, decap)	Weight & KV-cache tiling across banked memory	ECC, DBI metadata, & scrub share bank rows
Signal integrity: crosstalk & SSN on wide parallel bus	★ **Redundancy without breaking channel symmetry**	Refresh scheduling vs. compute pipeline bubbles
★ **I/O power: no burst structure $\Rightarrow$ no PHY DBI**	LPDDR5X as secondary tier: data placement policy	Known-good-die testing $\rightarrow$ post-package repair flow
μBump yield & alignment at 36 μm pitch	Compiler support for stream-aware access patterns	Multi-high stacking: routing, power, thermals
Clock distribution & skew across die stack	Runtime KV-cache alloc & eviction across banks	Hybrid bonding roadmap: $<$2 μm pitch, 8-high

★ The **challenges** we discuss in this talk

These Challenges Are Entangled



These Challenges Are Entangled



A solution to any one challenge **constrains the design space of the other two.**

Challenge 1: Bank Mapping

Challenge 1: The Bank-to-Channel Mapping Problem

The setup

- Each TE needs a **128 B flit** per access
- 16 ch × 16 TE-Groups = **256 ch/chiplet**
- Each bank delivers **32 B** per column access

Challenge 1: The Bank-to-Channel Mapping Problem

The setup

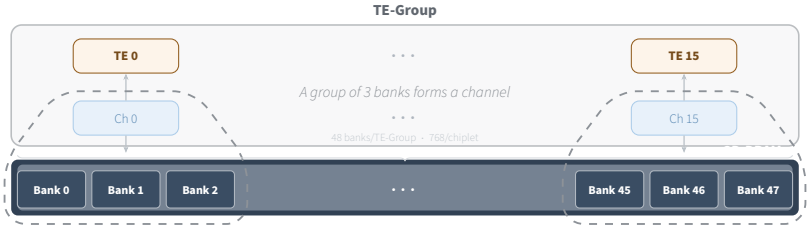
- Each TE needs a **128 B flit** per access
- 16 ch × 16 TE-Groups = **256 ch/chiplet**
- Each bank delivers **32 B** per column access

The arithmetic

- 128 B ÷ 32 B = **4** banks/ch *needed*
- DRAM die: **840 banks** (768 after 72 spares)
- 768 ÷ 256 = **3** banks/ch *available*

Need 4 banks/channel ⇒ have only 3 ⇒ **the 128 B flit does not divide evenly.**

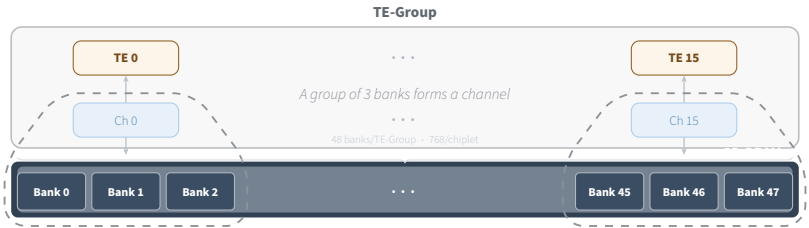
Challenge 1: The Overfetch Dilemma



With 3 banks per channel:

- One channel access returns **96 B** (3×32 B)
- 128 B flit $\Rightarrow$ two accesses $\Rightarrow$ 192 B fetched
- **33% overfetch $\approx$ 33 TB/s wasted**

Challenge 1: The Overfetch Dilemma



With 3 banks per channel:

- One channel access returns **96 B** (3 × 32 B)
- 128 B flit ⇒ two accesses ⇒ 192 B fetched
- **33% overfetch ≈ 33 TB/s wasted**

Naïve fix — column staggering:

- Stagger column indices to pack one flit per pair
- Requires a **192 B shifting buffer**
- Complicates timing closure and verification

How to deliver 128 B from **3** × **32** B banks **without wasting bandwidth or adding complex datapaths?**

Challenge 2: I/O Power at Scale

Challenge 2: The I/O Power Wall

The power cost of 100 TB/s:

Measured I/O Power

$P_{I/O} = 100$ TB/s $\times$ 0.35 pJ/bit $=$ **280** W

Why conventional DBI cannot help:

- HBM uses multi-cycle bursts (BL 4–16)
- PHY sees full burst $\Rightarrow$ decides on inversion
- DBI pin per byte lane signals choice

Our 3D-DRAM has **none of these**: single-cycle transfer, no burst, no sideband pin.

Challenge 2: The I/O Power Wall

The power cost of 100 TB/s:

Measured I/O Power

$P_{I/O} = 100$ TB/s $\times$ 0.35 pJ/bit $=$ **280** W

Why conventional DBI cannot help:

- HBM uses multi-cycle bursts (BL 4–16)
- PHY sees full burst $\Rightarrow$ decides on inversion
- DBI pin per byte lane signals choice

Our 3D-DRAM has **none of these**: single-cycle transfer, no burst, no sideband pin.

Interface comparison:

	DDR / HBM
Transfer	Multi-cycle burst
Lookahead	Full burst visible
DBI	Dedicated pin/lane

	Our 3D-DRAM
Transfer	Single-cycle 256-bit
Lookahead	**None**
DBI	**No pin. No sideband.**

Challenge 2: The I/O Power Wall

The power cost of 100 TB/s:

Measured I/O Power

$P_{I/O} = 100 \text{ TB/s} \times 0.35 \text{ pJ/bit} = \mathbf{280} \text{ W}$

Why conventional DBI cannot help:

- HBM uses multi-cycle bursts (BL 4–16)
- PHY sees full burst ⇒ decides on inversion
- DBI pin per byte lane signals choice

Our 3D-DRAM has **none of these**: single-cycle transfer, no burst, no sideband pin.

Interface comparison:

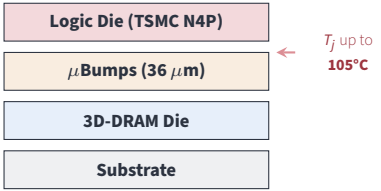
	DDR / HBM
Transfer	Multi-cycle burst
Lookahead	Full burst visible
DBI	Dedicated pin/lane

	Our 3D-DRAM
Transfer	Single-cycle 256-bit
Lookahead	**None**
DBI	**No pin. No sideband.**

DBI would save 20% more power. How to achieve DBI **architecturally?**

Challenge 3: Reliability Under Thermal Stress

Challenge 3: DRAM Reliability at 105°C



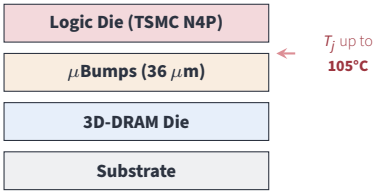
Retention consequence

Standard: 32 ms at 85°C

At 105°C: drops to **4 ms**

$\Rightarrow$ **8**$\times$ more frequent refresh

Challenge 3: DRAM Reliability at 105°C



Retention consequence

Standard: 32 ms at 85°C

At 105°C: drops to **4 ms**

⇒ **8**× more frequent refresh

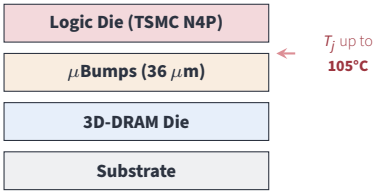
Three compounding problems:

1. Yield — 832 banks/die. Even a 1% fault rate risks losing entire channels.

2. Symmetry — Disabling a faulty bank narrows its channel. One weak channel throttles the entire slice.

3. Error accumulation — Higher T_j drives more soft errors. ECC, scrub, and refresh must all coexist without stalling throughput.

Challenge 3: DRAM Reliability at 105°C



Three compounding problems:

1. Yield — 832 banks/die. Even a 1% fault rate risks losing entire channels.

2. Symmetry — Disabling a faulty bank narrows its channel. One weak channel throttles the entire slice.

3. Error accumulation — Higher T_j drives more soft errors. ECC, scrub, and refresh must all coexist without stalling throughput.

Retention consequence

Standard: 32 ms at 85°C

At 105°C: drops to **4 ms**

$\Rightarrow$ **8**× more frequent refresh

How to maintain full-bandwidth, symmetric channels when banks fail and refresh is **8**× nominal?

Challenge 3: The Redundancy Topology Trap

✗ Edge-Placed Spare Banks

Reroute faults to spare banks at the die periphery.

- Long cross-die wires add latency
- Extra metal and routing power
- Defeats the benefit of short 3D vertical links

Impractical

Latency and power overhead negate 3D stacking gains.

Challenge 3: The Redundancy Topology Trap

✗ Edge-Placed Spare Banks

Reroute faults to spare banks at the die periphery.

- Long cross-die wires add latency
- Extra metal and routing power
- Defeats the benefit of short 3D vertical links

Impractical

Latency and power overhead negate 3D stacking gains.

✗ Skip Faulty Banks

Disable bad banks in place.

- Channels become asymmetric
- Tensor engines expect *uniform* widths
- One weak channel throttles the entire slice

Unrealistic

Bandwidth loss scales with fault count across the chiplet.

Need: Tolerate arbitrary faults, keep routing local, preserve symmetric channels.

Challenge 3: The Redundancy Topology Trap

✗ Edge-Placed Spare Banks

Reroute faults to spare banks at the die periphery.

- Long cross-die wires add latency
- Extra metal and routing power
- Defeats the benefit of short 3D vertical links

Impractical

Latency and power overhead negate 3D stacking gains.

✗ Skip Faulty Banks

Disable bad banks in place.

- Channels become asymmetric
- Tensor engines expect *uniform* widths
- One weak channel throttles the entire slice

Unrealistic

Bandwidth loss scales with fault count across the chiplet.

Need: Tolerate arbitrary faults, keep routing local, preserve symmetric channels.

Redundancy, refresh, and ECC share physical resources on every bank.
They must be co-designed.

Performance Results

Experimental Setup

Hardware configurations:

	BW	Capacity
SRAM	150 TB/s	4 GB
HBM	18 TB/s	192 GB
3D-DRAM	100 TB/s	32 GB

All configurations: 5 PFLOP/s tensor compute

Experimental Setup

Hardware configurations:

	BW	Capacity
SRAM	150 TB/s	4 GB
HBM	18 TB/s	192 GB
3D-DRAM	100 TB/s	32 GB

All configurations: 5 PFLOP/s tensor compute

Workloads and Deployment:

- **Dense LLMs:** Llama-3.1 70B, Llama-3.1 405B
- **MoE:** DeepSeek-V3 (671B), GPT-OSS (20B, 120B)
- **Speech:** Whisper, Canary-1B
- **Context Lengths:** 4K and 64K

Headline Results

4.71×

higher TPS/card
vs HBM

Headline Results

4.71×
higher TPS/card
vs HBM

2.44×
higher TPS/card
vs SRAM

Headline Results

4.71×
higher TPS/card
vs HBM

2.44×
higher TPS/card
vs SRAM

9.96×
lower TPOT
vs HBM

Headline Results

4.71×
higher TPS/card
vs HBM

2.44×
higher TPS/card
vs SRAM

9.96×
lower TPOT
vs HBM

Averaged across Llama-70B/405B, DeepSeek-V3, GPT-OSS, Whisper, and Canary.
At 4K context, batch size 32, 0.5 μs network latency, 1 TB/s network bandwidth.

Looking Forward

Roadmap: Hybrid Bonding

Current (F2F μBump):

Bonding pitch	36 μm
I/O energy	**0.35 pJ/bit**
DRAM stack	1-high
Bandwidth	**100 TB/s**/card
Capacity	**32 GB**/card

Today's Silicon

Channel Mapping + DBI +
thermal and redundancy features.
Production-validated at 105°C.

Roadmap: Hybrid Bonding

Current (F2F μBump):	
Bonding pitch	36 μm
I/O energy	**0.35 pJ/bit**
DRAM stack	1-high
Bandwidth	**100 TB/s**/card
Capacity	**32 GB**/card

Future (Hybrid Bonding):	
Bonding pitch	<2 μm
I/O energy	$\sim$**0.2 pJ/bit**
DRAM stack	8-high
Bandwidth	**800+ TB/s**/card
Capacity	**256 GB**/card

Today's Silicon

Channel Mapping + DBI +
thermal and redundancy features.
Production-validated at 105°C.

Roadmap: Hybrid Bonding

Current (F2F μBump):	
Bonding pitch	36 μm
I/O energy	**0.35 pJ/bit**
DRAM stack	1-high
Bandwidth	**100 TB/s**/card
Capacity	**32 GB**/card

Future (Hybrid Bonding):	
Bonding pitch	<2 μm
I/O energy	$\sim$**0.2 pJ/bit**
DRAM stack	8-high
Bandwidth	**800+ TB/s**/card
Capacity	**256 GB**/card

Today's Silicon
Channel Mapping + DBI + thermal and redundancy features. Production-validated at 105°C.

Projected Improvement
8$\times$ capacity, 8$\times$ bandwidth 7$\times$ lower energy per bit $\Rightarrow$ **56**$\times$ bandwidth-per-watt

Channel topology and architectural mechanisms $\Rightarrow$ **56$\times$ more bandwidth per watt.**

Conclusions

The problem:

- Generative AI decode is fundamentally memory-bound
- SRAM: bandwidth without capacity
- HBM: capacity without bandwidth

Results:

Metric	Value
TPS/card vs. HBM	**4.7**×
TPS/card vs. SRAM	**2.4**×
Responsiveness vs. HBM	**9.9**×
Bandwidth/card	**100 TB/s**
Capacity/card	**32 GB**
I/O energy	**0.35 pJ/bit**
Network sensitivity	Lower than SRAM
Roadmap (HB)	**56**× BW/W

Conclusions

The problem:

- Generative AI decode is fundamentally memory-bound
- SRAM: bandwidth without capacity
- HBM: capacity without bandwidth

Our contribution:

- 3D-DRAM closes the gap ⇒ bandwidth *and* capacity in one package
- Three co-designed solutions:
  stream blocking, stream flipping,
  topology-preserving redundancy
- Silicon-validated at 105°C

Results:

Metric	Value
TPS/card vs. HBM	**4.7**×
TPS/card vs. SRAM	**2.4**×
Responsiveness vs. HBM	**9.9**×
Bandwidth/card	**100 TB/s**
Capacity/card	**32 GB**
I/O energy	**0.35 pJ/bit**
Network sensitivity	Lower than SRAM
Roadmap (HB)	**56**× BW/W

Conclusions

The problem:

- Generative AI decode is fundamentally memory-bound
- SRAM: bandwidth without capacity
- HBM: capacity without bandwidth

Our contribution:

- 3D-DRAM closes the gap ⇒ bandwidth *and* capacity in one package
- Three co-designed solutions: stream blocking, stream flipping, topology-preserving redundancy
- Silicon-validated at 105°C

Results:

Metric	Value
TPS/card vs. HBM	**4.7**×
TPS/card vs. SRAM	**2.4**×
Responsiveness vs. HBM	**9.9**×
Bandwidth/card	**100 TB/s**
Capacity/card	**32 GB**
I/O energy	**0.35 pJ/bit**
Network sensitivity	Lower than SRAM
Roadmap (HB)	**56**× BW/W

The memory wall is a technology problem. **3D-DRAM is the solution.**

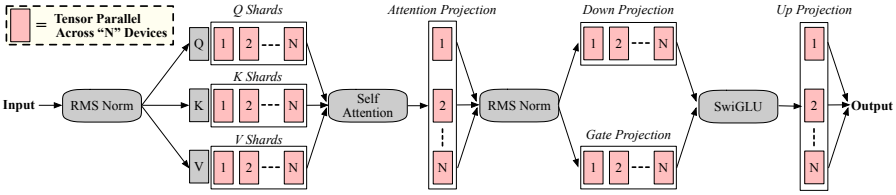
Scaling the Memory Wall for Generative Inference

Towards 3D-DRAM-based Accelerators

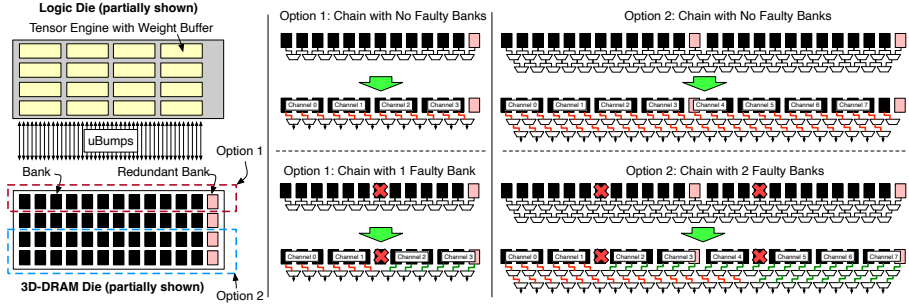
Thank You

Backup

Backup: Tensor Parallelism Detail

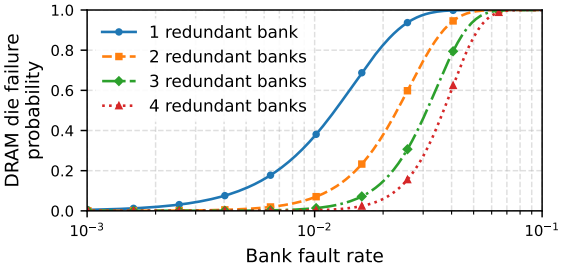


Backup: Bank Chaining for Yield



N functional + M redundant banks form a chain. Select any N-bank window to form channels. Tolerates M arbitrary faults while preserving full-width, symmetric channels.

Backup: Redundancy vs. Yield



Each additional redundant bank shifts the yield curve right, improving post-package yield.

Backup: Full Configuration Table

Config	Bandwidth	Capacity	Use
SRAM	150 TB/s	4 GB	Baseline
HBM	18 TB/s	192 GB	Baseline
3D-DRAM (Base)	100 TB/s	32 GB	Primary
3D-DRAM (2× BW)	200 TB/s	32 GB	Ablation
3D-DRAM (4× BW)	400 TB/s	32 GB	Ablation
3D-DRAM (2× Full)	200 TB/s	64 GB	Ablation
3D-DRAM (4× Full)	400 TB/s	128 GB	Ablation

Ablation variants isolate bandwidth vs. capacity effects.

Backup: Batch Size Analysis

(a) vs. Arrival Rate		(b) vs. Latency	
Req/s	**Max Batch**	**Latency (ms)**	**Rec. Batch**
110	1	0.1	1
200	3	1	1
500	5	2	1
1000	27	30	8
		100	16

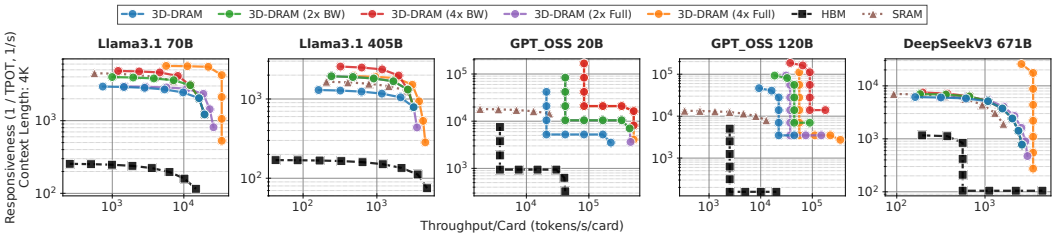
Poisson arrivals at production load ($\sim$9.5M req/day).
Batch size stays <32 even at high arrival rates $\Rightarrow$ practical operating point.

Backup: Deployment Configurations

Model	SRAM	HBM	3D-DRAM	Key Insight
Llama-70B	8 cards, TP8	1 card	4 cards, TP4	SRAM capacity-limited
Llama-405B	32 cards, PP32	4 cards, PP1	2 cards, PP2	3D-DRAM best balance
DeepSeek-V3	4+64 attn+exp	1+4	2+32	MoE disaggregated
GPT-OSS-20B	2+8	1+1	1+1	Small MoE fits
GPT-OSS-120B	8+32	1+1	1+4	Expert count matters
Whisper	1 card	1 card	1 card	Single-card, BW-bound
Canary-1B	1 card	1 card	1 card	Single-card, BW-bound

Key: SRAM needs many cards (capacity). HBM has capacity but starves (bandwidth).
3D-DRAM balances both.

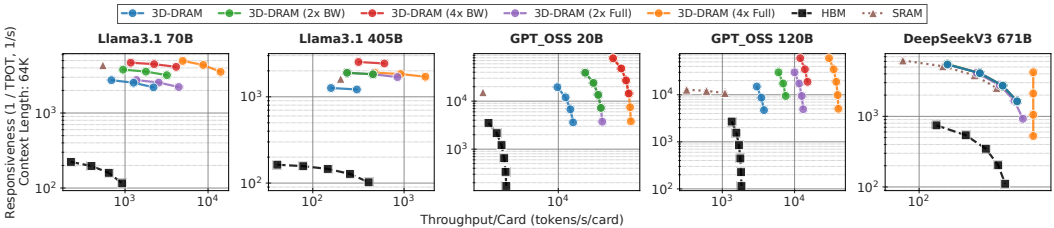
Responsiveness vs. Throughput (4K Context)



Setup: Minimal-card deployment, 0.5 μs latency, 1 TB/s network. Moving right = increasing batch size.

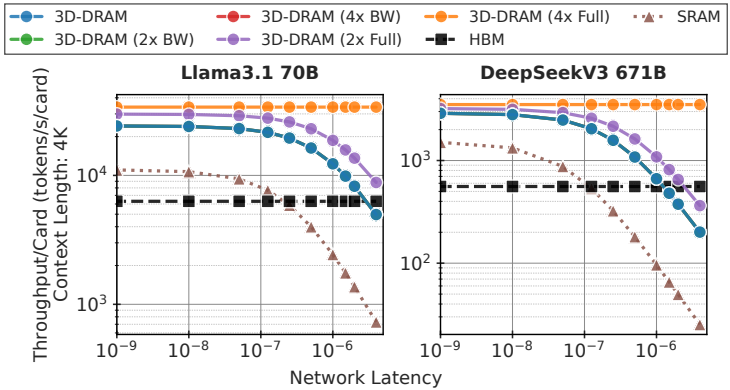
3D-DRAM dominates the upper-right (high throughput, high responsiveness).

Responsiveness vs. Throughput (64K Context)



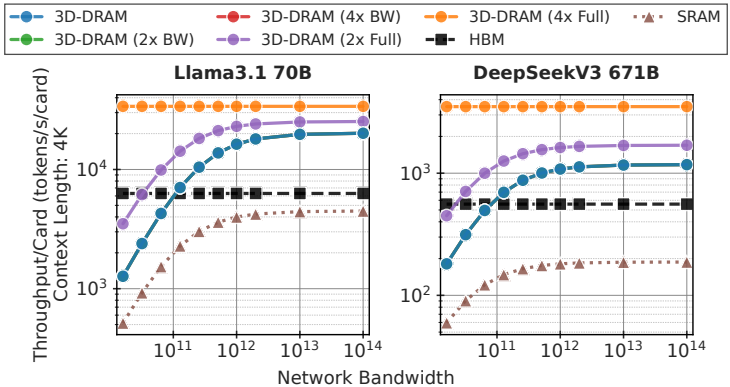
Long-context insight: KV cache grows 16×, pushing more systems into the memory-bound regime. 3D-DRAM's advantage grows with context length.

Network Latency Sensitivity



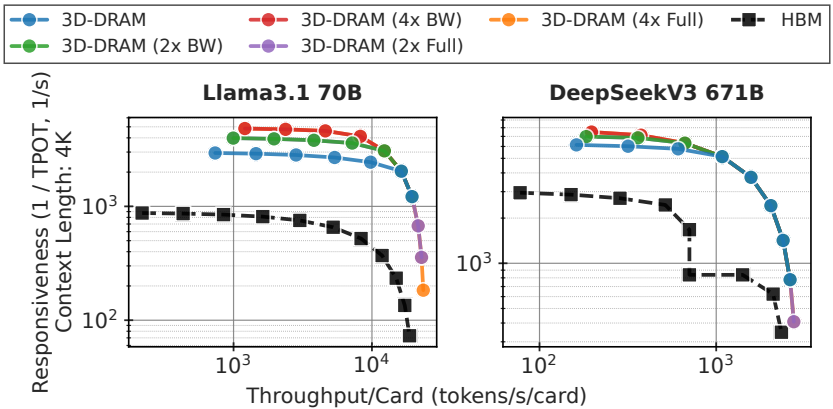
Key observations: TPS/card flat for latency $\leq 0.1\ \mu$s. SRAM most sensitive (high TP = many collectives).
Target link latency $\leq 0.5\ \mu$s to avoid the communication-dominated regime.

Network Bandwidth Sensitivity



Key observations: Dense LLMs saturate at BW $\geq$1 TB/s. MoE models saturate at 256 GB/s.
HBM relatively insensitive (already internally bandwidth-limited).

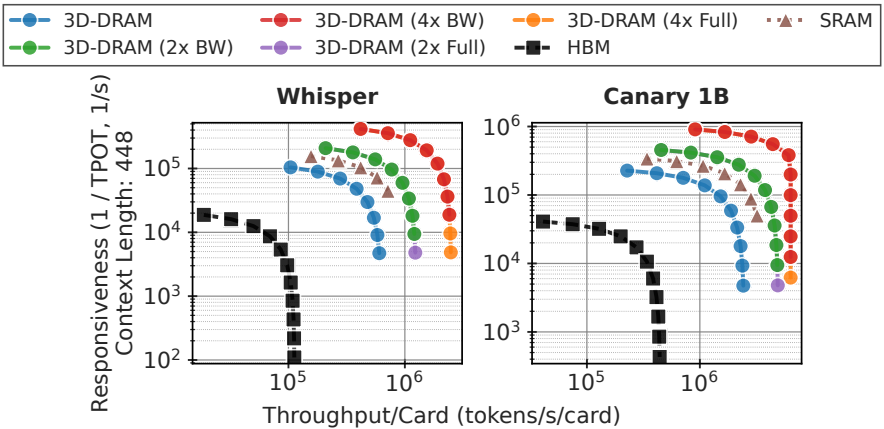
Iso-Card Comparison



Fair comparison: Fix card count to 3D-DRAM's minimal configuration. HBM can run larger batches but remains BW-limited.

Memory bandwidth dominates until compute saturation.

Speech Models: Single-Card Workloads



Key insight: Small models fit on a single card for all technologies ⇒ pure bandwidth comparison.
Ranking: SRAM > 3D-DRAM > HBM (directly tracks bandwidth: 150 > 100 > 18 TB/s).

Backup: What We Didn't Cover

Other memory system challenges (future work):

- Row buffer management policies
- Address interleaving schemes
- ECC design and coverage trade-offs
- Scrubbing schedules and bandwidth allocation
- Power delivery network through TSV arrays
- Signal integrity on dense μbump interfaces
- Test and known-good-die screening flows

Software co-optimization opportunities:

- KV cache placement aware of bank topology
- Weight layout matching stream blocking patterns
- Prefetch scheduling exploiting row buffer locality